

NOI2026 模拟赛 Day5 题目解析

云浅

2026-7-13

Peking University

1. 运输网络	2
2. 序列还原	8
3. 树的绘制	15

1. 运输网络

1. 运输网络

给定一棵 n 个点的树，树上的第 i 条边为 (u_i, v_i) 。再给定一个 $n \times n$ 的矩阵 A 。你需要求出一个 $1 \sim n$ 的排列 $p(1), p(2), \dots, p(n)$ ，最小化

$$\sum_{i=1}^{n-1} A_{p(u_i), p(v_i)}$$

的值。保证 $A_{i,i} = 0, A_{i,j} = A_{j,i}$ 。

此外，你还要求出有多少种不同的排列 p 能够最小化这个值。

数据范围： $1 \leq n \leq 18, 0 \leq A_{i,j} \leq 10^7$

1.1. 算法一

直接 $O(n! \times n)$ 暴力枚举所有排列并计算。

可以通过 Subtask1, 获得 8 分。

1.2. 算法二

考虑在树上 DP，设 $f(u, j, S)$ 表示：在 u 的子树内填数，所用的数恰好为集合 S ， $p(u) = j$ 的最小代价。转移时，枚举 u 的儿子 v 子树内填的集合 T ，以及 $p(v) = k$ ，做转移 $f(u, j, S) + f(v, k, T) + A_{j,k} \rightarrow f(u, j, S \cup T)$ 。

时间复杂度： $O(n^2 3^n)$ 或 $O(n^3 3^n)$ ，可以获得至少 48 分。由于实际状态数没有这么多，精细地实现（例如只保留 $|S| = \text{size}(u)$ 的状态）可以获得更高的分数。

1.3. 算法三

对于树是一条链的情况，可以直接从前往后 DP，记录最后一个数以及当前填入的数的集合，转移枚举下一个数填什么。时间复杂度 $O(2^n n^2)$

1.4. 算法四

考虑求出一个 DP 加点的顺序 $p_1, \dots, p_n$, 使得对任意时刻需要记录的点数不多。具体地, 我们希望能够找到一个排列 p , 使得对于一个 (较小的) 正整数 k , 满足每个前缀 $\{p_1, \dots, p_i\}$ 中, 在 $\{p_{i+1}, \dots, p_n\}$ 中存在相邻点的点个数 $\leq k$ 。

例如, 对任意的一棵树, 我们都能取到 $k = O(\log n)$: 轻重链剖分, 然后按照先访问轻儿子、再访问重儿子的顺序取一个 DFS 序即可。

如果能找到这样的排列, 那么我们就可以在 $O(2^n n^{k+1})$ 时间内解决: 按照这个顺序状压 DP, 记录后续可能相邻的点填入了什么数即可。

那么按照之前所述可以容易构造出 $k \leq 4$ 的顺序, 可以做到 $O(2^n n^5)$ 。进一步, 求解最小化 k 的排列也可以通过状压 DP 在 $O(2^n n)$ 时间内求出, 实践后可以发现所有不超过 18 个点的树都可以取到 $k \leq 2$, 那么可以在 $O(2^n n^3)$ 内解决本题, 期望得分 100。

事实上, 这是已经被证明的结论。上述最小的 k 称为这棵树的 pathwidth, 一般地, 对于 $k \geq 1$, 具有 pathwidth 为 k 的树的最小顶点数为 $\frac{5 \cdot 3^{k-1} - 1}{2}$ 。

2. 序列还原

2. 序列还原

有一个**元素互不相同**的非负整数序列 a ，其长度为 n 。保证 $0 \leq a_i \leq 10^9$ 。

你不知道序列 a 的具体内容。你可以多次进行以下两种询问之一：

1. 给出 l, r ，得到 $(\sum_{k=l}^r a_k) - (\min_{k=l}^r a_k)$ 。
2. 给出 k ，得到 a_k 的值。这种询问**最多只能进行两次**。

请使用不超过 $10^6 + 150$ 次询问 1，求出序列的具体内容。

数据范围： $n = 10^3$ 或 $n = 10^6$ 。

2.1. 如果知道 $\min$ 的位置

假如我们已经知道位置 p , 使得 $a_p = \min_{i=1}^n a_i$, 那么我们就可以询问某个 $[p, k]$ 和 $[p, k+1]$, 此时我们知道答案一定分别是 $\sum_{i=p}^k a_i - a_p$ 和 $\sum_{i=1}^{k+1} a_i - a_p$ 。二者相减即可得到 a_k 的值。

因此, 在知道最小值的位置之后, 只需要用 $n-1$ 次询问 1 即可求出整个序列。于是, 我们只需要用 151 次询问 1 和 2 次询问 2 求出最小值及其位置即可。

2.2. 算法零

我们不妨考虑 $n = 3$ 的情况，此时我们可以获得的所有信息是：

- $\text{ans}_{1,2} = \max(a_1, a_2)$
- $\text{ans}_{2,3} = \max(a_2, a_3)$
- $\text{ans}_{1,3} = a_1 + a_2 + a_3 - \min(a_1, a_2, a_3)$

由于我们有两次单点查询，所以只要能确定某一个 a_i 一定不是全局最小值，那么对剩下的两个数单点查询后比较一下即可知道全局最小值。

2.2. 算法零

可以用如下的方式确定：

- 若 $\text{ans}_{1,2} = \text{ans}_{2,3}$ ，说明 a_2 是 $\max$ ，可以排除 a_2 。
- 若 $\text{ans}_{1,2} > \text{ans}_{2,3}$ ，那么一定有 $a_1 > a_2$ （否则 $\text{ans}_{1,2} = a_2 \leq \max(a_2, a_3) = \text{ans}_{2,3}$ ），可以排除 a_1 。
- 若 $\text{ans}_{1,2} < \text{ans}_{2,3}$ ，可以排除 a_3 。

现在我们解决了 $n = 3$ 的情况，并且只用到 $\text{ans}_{1,2}$ 和 $\text{ans}_{2,3}$ 。

2.3. 算法一

一般地，我们考虑这样一个询问的 pattern: $f(a, c) - f(a, b) - f(b + 1, c)$ ，其中 $f(l, r)$ 表示询问区间 $[l, r]$ 的答案， $a < b < c$ 。

我们发现，它返回的就是 $\max(\min([a, b]), \min([b + 1, c]))$ 。

那么这就启发我们可以仿照 $n = 3$ 的情况，把序列平均分成三部分，假设三部分的 min 分别为 x_1, x_2, x_3 ，然后类似 $n = 3$ 的情况可以确定某个 x_i 一定不是最小值，这样就可以删掉这一段。

这样每次可以把 n 变成 $\frac{2}{3}n$ ，每次需要 $[a, b], [b + 1, c], [c + 1, d], [a, c], [b + 1, d]$ 询问五次，询问次数为 $5\lceil \log_{1.5} n \rceil$ ，在 $n = 10^6$ 时略微超过 150，可以获得 Sub1 满分和 Sub2 80% 分数，即 86 分。

实现的时候可以直接用 `vector` 来暴力维护当前还没被删除的所有位置，时间复杂度也是 $T(n) = T(\frac{2}{3}n) + O(n) = O(n)$ 的。询问的区间可能不连续，但被删除的位置一定不是 min，不会影响答案。

2.4. 算法二

进一步我们发现上述询问只需要比较以下两个数：

- $f(a, c) - f(a, b) - f(b + 1, c)$
- $f(b + 1, d) - f(b + 1, c) - f(c + 1, d)$

发现他们都减了一个 $f(b + 1, c)$ ，所以不需要询问 $[b + 1, c]$ ，也可以得到比较的结果。

这样询问次数就变为 $4\lceil \log_{1.5} n \rceil$ ，可以获得满分。

3. 树的绘制

3. 树的绘制

给定平面上的 n 个点，和一棵大小为 n 的树 T ，树上节点按 $1 \sim n$ 编号。

你需要为平面上的点使用 $1 \sim n$ 的编号重编号之后，对于所有树上的边 $e = (u, v)$ ，将平面上的点 u 和平面上的点 v 用线段连接后，任意两条线段除了在端点上相交没有其他的相交点。

试构造一组方案。输入数据保证一定有解。

数据范围： $1 \leq n \leq 2 \times 10^5$ 。

3.1. 算法一

如果所有点构成一个凸多边形，假设凸多边形上的点按顺序分别是 $P_1, \dots, P_n$ ，我们让 P_1 为根，然后按照子树大小把凸包划分为若干个区间，在每个区间内递归构造即可。

构造部分的时间复杂度是 $O(n)$ 的，给凸包上的点排序需要 $O(n \log n)$ 时间，总时间为 $O(n \log n)$ 。期望得分 20

3.2. 算法二

考虑以下的构造方案：选择 x 坐标最小的点为根，然后以这个点为中心极角排序，按极角序划分子树，每棵子树选择极角序第一个点为根，递归构造。

这样时间复杂度是 $O(n^2 \log n)$ 或 $O(n^2)$ 的（如果使用 `nth_element` 而不是每次排序），可以通过 Subtask 2，结合算法一可以获得 40 分。

3.3. 算法三

考虑链分治，取出根到叶子的一条重链 $\text{root} \rightarrow x$ ，我们把 root 和 x 放到两个点 A, B 上，使得所有点都在直线 AB 的同一侧（即， AB 是凸包上的相邻两个点）。

取出这条重链的中点 z ，我们考虑如果能够把 z 对应到合适的点 C ，使得能够从 C 这里把剩下的点分成两部分，在大小合适的前提下还能满足两侧都是凸的，就可以递归到 $\text{root} \rightarrow z$ 和 $z \rightarrow x$ 两部分。

那么算出两侧所需的点数 c_1, c_2 ，取出后 $c_2 + 1$ 个点中和 AB 夹角最小的一个即可。递归边界是重链就剩一个点了，此时就可以进入内部的子树处理别的重链。

根据实现方式，复杂度为 $O(n \log^2 n)$ 或 $O(n \log n)$ 。